CORS

Cross-Origin Resource Sharing

O que é CORS

Cross-Origin Resource Sharing (compartilhamento de recursos entre origens).

Um mecanismo do **navegador**, baseado em **headers HTTP**, que define se uma página de uma origem pode acessar recursos de outra origem.

ANALOGIA: O PORTEIRO DO PRÉDIO

Navegador	Servidor da API	CORS
O porteiro, que confere quem está entrando	O morador, que decide quem está autorizado	A lista de visitantes liberados que o morador entrega ao porteiro

O que é uma origem

`https://app.com:443`

Origem = **protocolo** + **domínio** + **porta**. Se qualquer um dos três mudar, é outra origem.

COMPARANDO COM HTTPS://APP.COM

`https://app.com/contato`	**Mesma origem**	Só o caminho mudou
`http://app.com`	**Outra origem**	Protocolo diferente (http)
`https://api.app.com`	**Outra origem**	Subdomínio diferente
`https://app.com:8080`	**Outra origem**	Porta diferente

Por que o navegador bloqueia

Por padrão, vale a **Same-Origin Policy**: uma página só pode ler respostas da **própria origem**.

SEM ESSA REGRA

Você abre um site malicioso, logado no seu banco em outra aba. Esse site faria requisições ao banco usando o seu login e leria a resposta.

> A Same-Origin Policy protege o **usuário**. O CORS é a forma de flexibilizá-la com autorização do servidor.

Para que serve

Permite que o **servidor diga ao navegador** quem pode ler as suas respostas.

01 Separar front-end e back-end em endereços diferentes

02 Abrir uma API pública para sites de terceiros

03 Escolher quais origens, métodos e headers são aceitos

04 Manter a proteção do usuário, liberando só o necessário

O erro que você vai encontrar

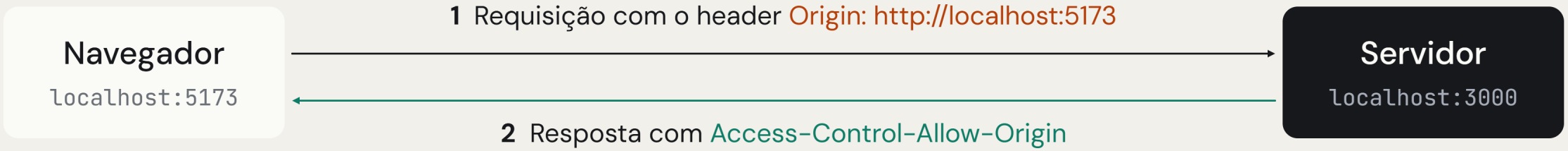
Front-end em localhost:5173 chamando a API em localhost:3000:

CONSOLE DO NAVEGADOR

```
Access to fetch at 'http://localhost:3000/usuarios' from origin 'http://localhost:5173' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

> A API não está quebrada. O navegador **bloqueou a leitura** da resposta porque o servidor não autorizou aquela origem.

Como funciona



3 · ORIGEM CONFERE

O navegador entrega a resposta ao seu código JavaScript.

3 · ORIGEM NÃO CONFERE

O navegador bloqueia a leitura e mostra o erro de CORS no console.

Os headers do CORS

`Origin`	Requisição	De onde a página está chamando (o navegador envia sozinho)
`Access-Control-Allow-Origin`	Resposta	Qual origem pode ler a resposta
`Access-Control-Allow-Methods`	Resposta	Métodos aceitos (GET, POST, PUT, DELETE…)
`Access-Control-Allow-Headers`	Resposta	Headers aceitos (Authorization, Content-Type…)
`Access-Control-Allow-Credentials`	Resposta	Permite enviar cookies e credenciais
`Access-Control-Max-Age`	Resposta	Por quantos segundos guardar o preflight

Preflight

Em requisições “não simples”, o navegador **pergunta antes** de enviar a requisição de verdade, usando o método OPTIONS.

01

Navegador envia OPTIONS: “posso fazer um DELETE com Authorization?”

02

Servidor responde com os headers Allow-Origin, Allow-Methods e Allow-Headers

03

Se for permitido, o navegador envia a requisição real

QUANDO ACONTECE

Métodos como **PUT**, **PATCH** e **DELETE**

Headers como **Authorization**

Body em JSON (**Content-Type: application/json**)

Quando usar

Precisa configurar

Front-end e API em origens diferentes (por exemplo, localhost:5173 e localhost:3000)

Front-end em um domínio e API em outro (app.com e api.com)

API pública consumida por sites de terceiros

Subdomínios diferentes chamando a mesma API

Não precisa

Front-end e API na mesma origem

Chamadas de um servidor para outro (back-end com back-end)

Postman, Insomnia e curl: não são navegadores

Apps nativos de celular

Liberando CORS no Express

1 · INSTALAR

```
npm install cors
```

O pacote **cors** é um middleware que adiciona os headers certos nas respostas.

2 · USAR NO SERVER.JS

```
const express = require("express");

const cors = require("cors");



const app = express();


app.use(cors());
app.use(express.json());
```

> Sem parâmetros, cors() libera **qualquer origem**. Serve para estudar, mas não para produção.

Configuração restrita

```
app.use(cors({
  origin: [
    "http://localhost:5173",
    "https://meusite.com"
  ],
  methods: ["GET", "POST", "PUT", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"],
  credentials: true
}));
```

`origin`
Só essas origens podem ler as respostas

`methods`
Métodos aceitos

`allowedHeaders`
Headers que o front pode enviar

`credentials`
Permite cookies e credenciais

Cuidados

Evite o asterisco em produção

origin: "*" libera qualquer site. Liste as origens que você realmente usa.

Credenciais exigem origem exata

Com credentials: true o navegador não aceita “*”. Informe a origem específica.

A origem precisa ser idêntica

Protocolo, domínio e porta. Barra no final ou http no lugar de https já causam erro.

Configure antes das rotas

O app.use(cors()) vem antes das rotas, para valer também para o preflight.

CORS protege a minha API?

Não.

O CORS é uma regra que **só o navegador** aplica. Quem faz a requisição por outro caminho ignora o CORS por completo.

NÃO É

Autenticação

Autorização

Firewall ou proteção da API

É

Uma proteção do **usuário** dentro do navegador

Uma lista de origens autorizadas pelo servidor

Complementar ao token e à validação no servidor

Como resolver o erro

01 Abra a aba **Network** do navegador e veja a requisição que falhou

02 Confira o header Origin enviado e compare com o que o servidor libera

03 Se houver uma requisição OPTIONS, veja se ela respondeu com sucesso e com os headers certos

04 Ajuste o **servidor**: origem, métodos e headers permitidos

05 Reinicie a API e teste de novo, com o cache do navegador limpo

Lembrete: a correção quase sempre é no servidor. Mexer no front-end não resolve.

PRÁTICA

Vamos provocar o erro de CORS e liberar o nosso front-end

`npm install cors` `app.use(cors())` `origin`